

上海财经大学

信息管理与工程学院

整数规划进阶建模与求解

优化理论与算法 典型模型结构与求解直觉

王震

2026

主要知识点

知识模块	主要内容
典型 0-1 模型结构	集合覆盖、集合划分、集合打包，以及它们在车辆路线选择、存货切割（下料）和广告档期中的应用
逻辑与线性化技巧	至少 K 个条件成立、多个条件推出结论、有限取值函数、乘积与分段线性函数线性化
时间维度建模	生产计划、库存平衡、固定开机成本、人员排班和时间索引变量
求解算法直觉	枚举复杂度、NP-hard、LP 松弛、MIP gap、分支定界、割平面、预处理、启发式和列生成
综合建模	将选址、分配、容量、覆盖、服务水平和启用关系组合成完整模型

典型结构一：集合模型快速回顾

模型	约束形式	典型含义
集合覆盖	$\sum_j a_{ij}x_j \geq 1$	每个需求至少被满足一次
集合划分	$\sum_j a_{ij}x_j = 1$	每个任务恰好被一个方案处理
集合打包	$\sum_j a_{ij}x_j \leq 1$	每个资源至多被占用一次

共同表达

a_{ij} 是已知参数，表示方案 j 是否覆盖或占用元素 i ； x_j 是是否选择方案 j 的决策变量。

集合划分问题

集合划分要求每个元素恰好被一个方案覆盖。

$$\begin{aligned} \min \quad & \sum_{j \in J} c_j x_j \\ \sum_{j \in J} a_{ij} x_j &= 1, \quad \forall i \in I, \\ x_j &\in \{0, 1\}, \quad \forall j \in J. \end{aligned}$$

典型场景

航班机组排班、车辆路径选择、订单打包、任务分组。每个任务既不能遗漏，也不能重复安排。

应用案例：车辆路线选择

配送平台有订单集合 I ，车辆从仓库出发并返回仓库。直接决定车辆行驶弧会遇到容量、时间窗、访问顺序和子回路等约束。

集合建模视角

先把一条完整路线看成一个候选方案 $r \in R$ 。每条候选路线已经满足车辆容量、时间窗、工作时长和路径连通性等内部规则。

符号	含义
a_{ir}	给定参数；路线 r 服务订单 i 时为 1，否则为 0
c_r	路线 r 的运输成本或总行驶时间
x_r	决策变量；选择路线 r 时为 1，否则为 0

车辆路线选择：集合划分模型

若每个订单必须恰好由一条路线服务，且最多使用 K 辆车，可写为

$$\begin{aligned} \min \quad & \sum_{r \in R} c_r x_r \\ \text{s.t.} \quad & \sum_{r \in R} a_{ir} x_r = 1, \quad \forall i \in I, \\ & \sum_{r \in R} x_r \leq K, \\ & x_r \in \{0, 1\}, \quad \forall r \in R. \end{aligned}$$

约束解释

第一组约束是集合划分：不能遗漏订单，也不能让同一订单被两条路线重复服务。

车辆路线选择：难点在哪里

集合划分模型把路线 r 当作一个候选方案，因此主模型非常简洁。难点在于：一条“合法路线”本身已经包含许多业务规则。

问题层面	需要表达的规则
------	---------

路线内部	车辆从仓库出发并返回仓库；访问顺序形成一条连通路径；不超过车辆容量；满足客户时间窗；司机工作时长受限。
------	---

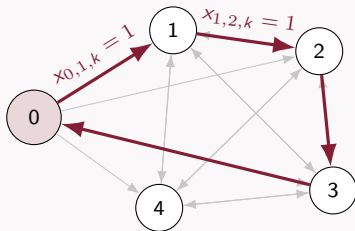
路线组合	每个订单恰好服务一次；车辆数量有限；不同路线组合后的总成本最小。
------	----------------------------------

核心思想

集合划分不是让问题消失，而是把复杂性拆成两部分：先生成合法路线，再选择路线组合。

车辆路线选择：原始弧变量模型

若直接建模，通常在节点集合 $N = \{0, 1, \dots, n\}$ 上定义弧变量。0 表示仓库， K 表示车辆集合。



若车辆 k 走弧 (i, j) , 则 $x_{ijk} = 1$

若车辆 k 服务客户 i , 则 $y_{ik} = 1$

图中红色弧表示一辆车
从仓库出发、访问客户、再返回仓库。

$$x_{ijk} = \begin{cases} 1, & \text{车辆 } k \text{ 从节点 } i \text{ 直接行驶到节点 } j, \\ 0, & \text{否则,} \end{cases}$$

$$y_{ik} = \begin{cases} 1, & \text{车辆 } k \text{ 服务客户 } i, \\ 0, & \text{否则.} \end{cases}$$

车辆路线选择：目标函数与第一层约束

一个基本目标函数可写为

$$\min \sum_{k \in K} \sum_{i \in N} \sum_{j \in N, j \neq i} c_{ij} x_{ijk}.$$

第一层约束

$$\sum_{k \in K} y_{ik} = 1, \quad \forall i = 1, \dots, n, \quad \sum_{j \neq 0} x_{0jk} \leq 1, \quad \forall k \in K.$$

每个客户恰好服务一次；每辆车至多从仓库出发一次。

车辆路线选择：连通性与容量约束

弧变量模型还必须保证：车辆进入某个客户，就必须从该客户离开。

$$\sum_{j \in N, j \neq i} x_{ijk} = y_{ik}, \quad \sum_{j \in N, j \neq i} x_{jik} = y_{ik}, \quad \forall i = 1, \dots, n, \forall k \in K.$$

若客户 i 的需求量为 d_i ，车辆容量为 Q ，则有

$$\sum_{i=1}^n d_i y_{ik} \leq Q, \quad \forall k \in K.$$

更细的容量表达

若需要描述车辆载重沿路线逐步变化，还要引入载重变量 q_{ik} ，并用 Big-M 约束刻画

$$q_{jk} \geq q_{ik} + d_j - M(1 - x_{ijk}).$$

这类约束把“访问顺序”和“车辆载重”绑定在一起。

车辆路线选择：时间窗与服务顺序

许多配送问题要求客户 i 在时间窗 $[a_i, b_i]$ 内开始服务。令 t_{ik} 表示车辆 k 到达客户 i 的时间。

$$a_i y_{ik} \leq t_{ik} \leq b_i y_{ik} + M(1 - y_{ik}), \quad \forall i, k.$$

若车辆从 i 到 j 的行驶时间为 τ_{ij} ，客户 i 的服务时间为 s_i ，则

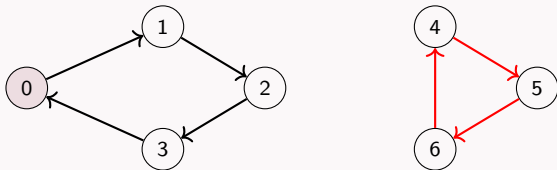
$$t_{jk} \geq t_{ik} + s_i + \tau_{ij} - M(1 - x_{ijk}), \quad \forall i \neq j, \forall k.$$

建模含义

当 $x_{ijk} = 1$ 时，时间顺序约束生效：服务完 i 并行驶到 j 之后，才能开始服务 j 。当 $x_{ijk} = 0$ 时，该约束被 Big-M 放松。

车辆路线选择：子回路消除

仅有进出平衡约束还不够，模型可能出现不经过仓库的客户小环路。



一种通用写法是对子集 $S \subseteq \{1, \dots, n\}$ 加约束：

$$\sum_{k \in K} \sum_{i \in S} \sum_{j \in S, j \neq i} x_{ijk} \leq |S| - 1, \quad \forall S, |S| \geq 2.$$

困难之处

这类约束数量随客户数指数增长，实际求解时通常按需加入，而不是一开始全部列出。

车辆路线选择：为什么改写成集合划分

原始弧变量模型的优点是直接、完整；缺点是变量和约束都很重，尤其在有时间窗、容量、工作时长和子回路约束时。

建模方式	主要变量	适合讲清楚的问题
弧变量模型	x_{ijk} : 车辆是否走弧 (i,j)	路线如何形成、容量和时间如何沿路线传播
集合划分模型	x_r : 是否选择完整路线 r	如何从大量合法路线中选出最优组合

算法连接

当路线集合 R 太大时，可以先放入一部分路线求主问题，再用定价子问题寻找新的低成本路线。这就是列生成和分支定价的基本思想。

整数规划应用：存货切割问题

造纸公司只生产长度为 25 米的原始纸卷，需要切割成 5 种固定长度的产品。

产品长度	5m	8m	12m	15m	17m
需求量	40	35	30	25	20

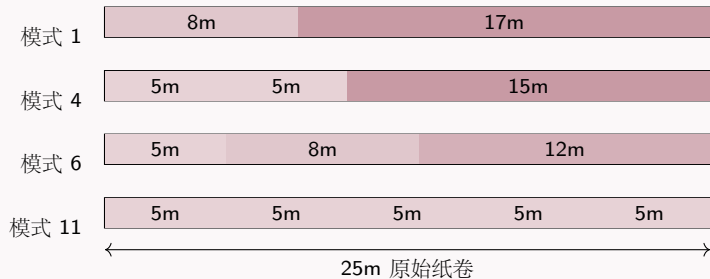
课堂思考

应该怎样定义决策变量？

- ▶ 方案 A：令变量表示每种产品最终生产多少件；
- ▶ 方案 B：令变量表示每一根原始纸卷具体如何切；
- ▶ 方案 C：令变量表示每一种切割模式使用多少次；
- ▶ 方案 D：令变量表示某种长度的产品是否生产。

存货切割问题：一根纸卷如何切

一根 25 米原始纸卷可以有多种切法。每一种切法称为一个切割模式。



观察：模式 1 表示切出 $8m + 17m$ ，模式 6 表示切出 $5m + 8m + 12m$ 。模型不直接决定每一刀的位置，而是决定每种模式使用多少根纸卷。

存货切割问题：建模思想

前一页的方案 C 是更自然的整数规划表达：先列出若干可行切割模式，再决定每种模式使用多少次。

切割模式

每个模式说明一根 25 米纸卷可以切出哪些产品。若第 j 个模式切出长度 i 产品的件数为 a_{ij} ，则 a_{ij} 是已知参数，不是决策变量。

$$\sum_i \ell_i a_{ij} \leq 25, \quad a_{ij} \in \mathbb{Z}_+.$$

建模转换

把“每根纸卷怎么切”的复杂选择，转换为“每种切割模式用多少次”的整数变量。

存货切割问题：11 种切割模式

产品长度	模式 1	模式 2	模式 3	模式 4	模式 5	模式 6	模式 7	模式 8	模式 9	模式 10	模式 11
5m	0	1	0	2	0	1	2	0	1	3	5
8m	1	0	1	0	0	1	0	3	2	1	0
12m	0	0	0	0	2	1	1	0	0	0	0
15m	0	0	1	1	0	0	0	0	0	0	0
17m	1	1	0	0	0	0	0	0	0	0	0

参数含义

表中数字 a_{ij} 是已知参数，表示模式 j 能切出长度 i 的产品件数；它不是决策变量。

存货切割问题：模型

令 $x_j \in \mathbb{Z}_+$ 表示第 j 种切割模式使用的纸卷数量。目标是使用尽可能少的 25 米原始纸卷。

$$\begin{aligned} \min \quad & \sum_{j=1}^{11} x_j \\ \text{s.t.} \quad & x_2 + 2x_4 + x_6 + 2x_7 + x_9 + 3x_{10} + 5x_{11} \geq 40, \\ & x_1 + x_3 + x_6 + 3x_8 + 2x_9 + x_{10} \geq 35, \\ & 2x_5 + x_6 + x_7 \geq 30, \\ & x_3 + x_4 \geq 25, \\ & x_1 + x_2 \geq 20, \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, 11. \end{aligned}$$

解释

五条需求约束分别对应 5m、8m、12m、15m 和 17m 产品。约束用“ $\geq$ ”表示产量至少满足需求。

集合打包问题

集合打包要求每个资源或元素至多被一个方案占用。

$$\begin{aligned} \max \quad & \sum_{j \in J} p_j x_j \\ \sum_{j \in J} a_{ij} x_j & \leq 1, \quad \forall i \in I, \\ x_j & \in \{0, 1\}, \quad \forall j \in J. \end{aligned}$$

典型场景

广告档期选择、会议室安排、直播排期、互斥项目组合。资源一旦被占用，不能重复使用。

案例：广告档期打包

平台有若干广告套餐，每个套餐占用若干时间段。若套餐 j 占用时间段 t ，则 $a_{tj} = 1$ 。

$$x_j = \begin{cases} 1, & \text{选择广告套餐 } j, \\ 0, & \text{不选择.} \end{cases}$$

每个时间段最多被一个套餐占用：

$$\sum_{j \in J} a_{tj} x_j \leq 1, \quad \forall t.$$

收益最大化：

$$\max \sum_{j \in J} p_j x_j.$$

练习 1：判断模型类型

请判断下列问题更接近哪一类集合模型。

1. 选择若干服务网点，使每个客户区至少有一个网点可达；
2. 选择若干配送路线，使每个订单恰好由一条路线服务；
3. 选择若干直播活动，使每个黄金时间段最多安排一个活动；
4. 选择若干巡检路线，使每个设备点至少被巡检一次。

练习 1：判断模型类型

请判断下列问题更接近哪一类集合模型。

1. 选择若干服务网点，使每个客户区至少有一个网点可达；
2. 选择若干配送路线，使每个订单恰好由一条路线服务；
3. 选择若干直播活动，使每个黄金时间段最多安排一个活动；
4. 选择若干巡检路线，使每个设备点至少被巡检一次。

答案

依次为：集合覆盖、集合划分、集合打包、集合覆盖。

拓展：覆盖强度与冗余覆盖

有时“至少覆盖一次”并不够，需要提高服务可靠性。例如重要区域至少由两个网点覆盖。

$$\sum_{j \in J} a_{ij} x_j \geq r_i, \quad \forall i \in I.$$

其中 r_i 是区域 i 的覆盖要求。

商业含义

核心商圈、重点客户或高风险区域可以设置更高覆盖强度，以提升服务韧性。

集合模型案例要点

- ▶ 车辆路线选择中，候选方案是一条完整合法路线，订单通常要求恰好服务一次；
- ▶ 存货切割问题中，候选方案是一种切割模式，产品需求通常要求至少满足；
- ▶ 广告档期选择中，候选方案会占用时间段，时间段通常要求至多占用一次；
- ▶ 真正的难点常常不在主约束，而在候选方案数量巨大；
- ▶ 这类问题自然引出列生成：先选已有方案，再寻找新的有价值方案。

典型结构二：逻辑建模工具箱

许多业务规则不是简单的容量或需求约束，而是“满足若干条件后，另一条规则才生效”。

问题：遇到复杂业务规则时，如何先判断它属于哪一类逻辑结构？

典型结构二：逻辑建模工具箱

许多业务规则不是简单的容量或需求约束，而是“满足若干条件后，另一条规则才生效”。

问题：遇到复杂业务规则时，如何先判断它属于哪一类逻辑结构？

规则类型	建模思想
至少 K 条规则成立	给每条规则配一个 0-1 变量，统计被激活的规则数量
多个条件推出一个结论	用 0-1 条件变量共同控制结论约束是否生效
约束可开可关	用 Big-M 或 indicator constraint 表达“激活时必须满足”
函数只能取有限个值	用 one-hot 变量选择一个取值或一个档位

共同原则：先明确 0-1 变量的含义，再写它如何控制约束。

用额外 0-1 变量控制约束是否生效

很多模型需要表达：只有启用某个方案时，某条约束才必须满足。

用额外 0-1 变量控制约束是否生效

很多模型需要表达：只有启用某个方案时，某条约束才必须满足。

处理技巧：令 $y \in \{0, 1\}$ 表示是否启用，并用 y 控制约束是否生效。

$$\mathbf{a}^\top \mathbf{x} \leq \mathbf{b} + M(1 - y).$$

情形	含义
$y = 1$	约束变为 $\mathbf{a}^\top \mathbf{x} \leq \mathbf{b}$ ，必须满足
$y = 0$	约束变为 $\mathbf{a}^\top \mathbf{x} \leq \mathbf{b} + M$ ，被放松

求解器视角：indicator constraint 可直接写成“若 $y = 1$ ，则 $\mathbf{a}^\top \mathbf{x} \leq \mathbf{b}$ ”。

至少 K 个约束成立

给定 N 个候选约束:

$$\mathbf{a}_i^\top \mathbf{x} \leq b_i, \quad i = 1, \dots, N.$$

要求其中至少 K 个约束成立。

至少 K 个约束成立

给定 N 个候选约束：

$$\mathbf{a}_i^\top \mathbf{x} \leq b_i, \quad i = 1, \dots, N.$$

要求其中至少 K 个约束成立。

处理技巧：令 $y_i \in \{0, 1\}$ 表示第 i 条约束被激活。

$$\mathbf{a}_i^\top \mathbf{x} \leq b_i + M_i(1 - y_i), \quad i = 1, \dots, N,$$

$$\sum_{i=1}^N y_i \geq K,$$

$$y_i \in \{0, 1\}, \quad i = 1, \dots, N.$$

解释：当 $y_i = 1$ 时，第 i 条约束必须成立；当 $y_i = 0$ 时，该约束被放松。

至少 K 个约束成立：业务例子

一个新店开业方案需要通过若干项评估：预算、客流、租金、人员、消防、交通。管理层规定：至少通过其中 4 项，方案才允许进入下一轮。

至少 K 个约束成立：业务例子

一个新店开业方案需要通过若干项评估：预算、客流、租金、人员、消防、交通。管理层规定：至少通过其中 4 项，方案才允许进入下一轮。

处理技巧：用 y_i 表示第 i 项评估是否被要求满足，并统计通过项数。

$$y_i = \begin{cases} 1, & \text{第 } i \text{ 项评估被要求满足,} \\ 0, & \text{第 } i \text{ 项评估可暂不满足.} \end{cases}$$

$$\text{评估约束 } i: \quad \mathbf{a}_i^\top \mathbf{x} \leq b_i + M_i(1 - y_i), \quad \sum_{i=1}^6 y_i \geq 4.$$

提醒： M_i 应按第 i 条约束的最大可能违反量分别取值。

多个条件同时成立后推出一个结论

设 $z_1, \dots, z_N \in \{0, 1\}$ 表示 N 个前提条件是否成立。若这些条件全部成立，则结论约束

$$\mathbf{a}^\top \mathbf{x} \leq b$$

必须成立。

多个条件同时成立后推出一个结论

设 $z_1, \dots, z_N \in \{0, 1\}$ 表示 N 个前提条件是否成立。若这些条件全部成立，则结论约束

$$a^\top x \leq b$$

必须成立。

处理技巧：用前提变量之和判断所有条件是否同时成立。

$$a^\top x \leq b + M \left(N - \sum_{i=1}^N z_i \right).$$

- ▶ 若 $z_1 = \dots = z_N = 1$ ，右侧为 b ，结论约束生效；
- ▶ 只要有一个前提不成立，右侧增加至少 M ，结论约束被放松。

例子：若“选择直播活动”“选择首页广告位”“选择周末档期”都成立，则平台审核成本不得超过预算上限。

有限取值函数建模

若某个变量或函数值只能从有限集合 $\{d_1, \dots, d_N\}$ 中选择一个，应该如何建模？

有限取值函数建模

若某个变量或函数值只能从有限集合 $\{d_1, \dots, d_N\}$ 中选择一个，应该如何建模？

处理技巧：引入 **one-hot** 变量 y_i ，用它选择唯一一个取值。

$$v = \sum_{i=1}^N d_i y_i, \quad \sum_{i=1}^N y_i = 1, \quad y_i \in \{0, 1\}.$$

业务含义	有限取值例子
价格档位	只能选择 9.9 元、12.9 元或 15.9 元
服务等级	标准、加急、专属三种等级
折扣政策	无折扣、九折、八折、满减四种规则

关键： $\sum_i y_i = 1$ 表示“恰好选择一个档位”；若允许不选，则改为 $\sum_i y_i \leq 1$ 。

更完整的乘积线性化

问题中若出现 $z = xy$ 、 $z = \prod_i x_i$ 或 $z = yu$ ，模型就不再是线性整数规划。

更完整的乘积线性化

问题中若出现 $z = xy$ 、 $z = \prod_i x_i$ 或 $z = yu$ ，模型就不再是线性整数规划。

处理技巧：引入辅助变量 z ，并用线性不等式刻画乘积关系。

非线性项	线性化方式
$z = xy, x, y \in \{0, 1\}$	$z \leq x, z \leq y, z \geq x + y - 1, z \geq 0$
$z = \prod_{i=1}^N x_i, x_i \in \{0, 1\}$	$z \leq x_i \ \forall i, \quad z \geq \sum_{i=1}^N x_i - (N - 1), \quad z \in \{0, 1\}$
$z = yu, y \in \{0, 1\}, 0 \leq u \leq U$	$z \leq Uy, z \leq u, z \geq u - U(1 - y), z \geq 0$

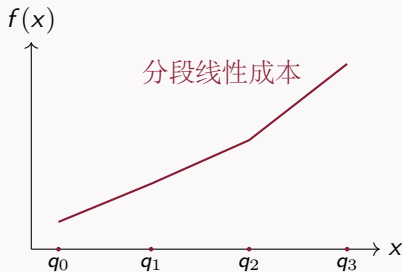
教学重点：线性化必须补足约束，使新变量 z 在所有情形下都等于原乘积。

分段线性函数：为什么需要额外变量

运营成本常常不是一条直线。例如订单量较小时使用人工处理，订单量较大时启用自动化设备，边际成本会变化。

分段线性函数：为什么需要额外变量

运营成本常常不是一条直线。例如订单量较小时使用人工处理，订单量较大时启用自动化设备，边际成本会变化。



建模思路

先判断 x 落在哪个区间，再使用该区间对应的线性表达式。

分段线性函数：区间选择建模

设 x 只能落在三个区间之一：

$$[L_1, U_1], \quad [L_2, U_2], \quad [L_3, U_3].$$

分段线性函数：区间选择建模

设 x 只能落在三个区间之一：

$$[L_1, U_1], \quad [L_2, U_2], \quad [L_3, U_3].$$

处理技巧：令 $w_i \in \{0, 1\}$ 表示是否选择第 i 个区间，并令 x_i 表示全局变量 x 在第 i 个区间中的复制值。

$$\begin{aligned} \sum_{i=1}^3 w_i &= 1, & x &= \sum_{i=1}^3 x_i, \\ L_i w_i &\leq x_i \leq U_i w_i, & i &= 1, 2, 3. \end{aligned}$$

令 v 表示函数值。若第 i 段函数为 $f_i(x) = \alpha_i x + \beta_i$ ，则

$$v = \sum_{i=1}^3 (\alpha_i x_i + \beta_i w_i).$$

解释：只有 $w_i = 1$ 的区间允许 $x_i = x$ ，其他 $x_i = 0$ 。若 x_i 定义为局部增量，则截距项需平移。

逻辑与线性化技巧小结

- ▶ “至少 K 个成立”本质是统计被激活约束的数量；
- ▶ “多个条件推出结论”可以用前提变量的和控制结论是否生效；
- ▶ 额外 0-1 变量常用于控制约束、选择档位、选择区间；
- ▶ 乘积线性化要保证辅助变量与原非线性项完全一致；
- ▶ 分段线性函数的关键是“先选区间，再写该区间的线性表达式”。

课堂提醒

这些技巧的共同目标不是增加变量，而是把业务规则转写成求解器能够处理的线性约束。

典型结构三：时间维度建模

带时间维度的问题常见于生产、库存、排班和调度。

- ▶ 每期生产多少；
- ▶ 是否在某期开机；
- ▶ 每个时间段需要多少员工；
- ▶ 每个任务在什么时间开始。

核心思想

用时间索引变量记录每个周期或时间段的决策，并用平衡约束或覆盖约束连接相邻时期。

生产计划的基本记号

设 $t = 1, \dots, T$ 表示生产周期。

记号	含义
x_t	第 t 期生产量
l_t	第 t 期末库存
d_t	第 t 期需求
c_t	单位生产成本
h_t	单位库存成本

关键

生产计划模型的核心不是目标函数，而是库存平衡约束。

库存平衡约束

库存平衡为

$$l_{t-1} + x_t = d_t + l_t, \quad t = 1, \dots, T.$$

流入

$$l_{t-1} + x_t$$

上一期末库存加本期生产。

流出

$$d_t + l_t$$

本期需求加本期期末库存。

该式也可写成 $l_t = l_{t-1} + x_t - d_t$ 。

基本生产计划模型

$$\begin{aligned} \min \quad & \sum_{t=1}^T c_t x_t + \sum_{t=1}^T h_t l_t \\ & l_{t-1} + x_t = d_t + l_t, \quad t = 1, \dots, T, \\ & 0 \leq x_t \leq U_t, \quad l_t \geq 0. \end{aligned}$$

解释

生产成本鼓励少生产，库存成本鼓励少积压，库存平衡保证每期需求被满足。

带固定开机成本的生产计划

若第 t 期是否开机需要决策, 引入

$$y_t = \begin{cases} 1, & \text{第 } t \text{ 期开机,} \\ 0, & \text{第 } t \text{ 期不开机.} \end{cases}$$

模型为

$$\begin{aligned} \min \quad & \sum_{t=1}^T F_t y_t + \sum_{t=1}^T c_t x_t + \sum_{t=1}^T h_t l_t \\ & l_{t-1} + x_t = d_t + l_t, \quad x_t \leq U_t y_t, \\ & x_t, l_t \geq 0, \quad y_t \in \{0, 1\}. \end{aligned}$$

开机约束的解释

约束

$$x_t \leq U_t y_t$$

是 Big-M 思想在生产计划中的自然形式。

不开机

$$y_t = 0 \Rightarrow x_t \leq 0.$$

因此 $x_t = 0$ 。

开机

$$y_t = 1 \Rightarrow x_t \leq U_t.$$

生产量受产能限制。

生产计划例题

某工厂未来 3 周需求为

$$d = (30, 50, 40).$$

每周产能上限为 60。若某周开机，需要固定成本 100；单位生产成本为 4；单位库存成本为 1。初始库存 $l_0 = 0$ 。

建模任务

定义 x_t, l_t, y_t ，写出三期库存平衡约束、开机约束和目标函数。

生产计划例题：模型答案

库存平衡：

$$x_1 = 30 + l_1,$$

$$l_1 + x_2 = 50 + l_2,$$

$$l_2 + x_3 = 40 + l_3.$$

开机约束：

$$x_1 \leq 60y_1, \quad x_2 \leq 60y_2, \quad x_3 \leq 60y_3.$$

目标函数：

$$\min 100(y_1 + y_2 + y_3) + 4(x_1 + x_2 + x_3) + (l_1 + l_2 + l_3).$$

拓展：最小生产批量

有些设备一旦开机，生产量不能太小。若最小批量为 L_t ，则

$$L_t y_t \leq x_t \leq U_t y_t.$$

不开机

$$y_t = 0 \Rightarrow 0 \leq x_t \leq 0.$$

开机

$$y_t = 1 \Rightarrow L_t \leq x_t \leq U_t.$$

这是固定成本模型的常见增强。

人员排班问题

排班问题的核心是让每个时间段的人手需求得到覆盖。

- ▶ $t \in T$: 时间段;
- ▶ $s \in S$: 可选班次;
- ▶ $a_{ts} = 1$: 班次 s 覆盖时间段 t ;
- ▶ d_t : 时间段 t 的人员需求;
- ▶ x_s : 安排到班次 s 的人数。

$$\sum_{s \in S} a_{ts} x_s \geq d_t, \quad \forall t \in T.$$

排班模型

若 c_s 是班次 s 的单位成本，则可写为

$$\min \sum_{s \in S} c_s x_s$$

$$\sum_{s \in S} a_{ts} x_s \geq d_t, \quad \forall t \in T,$$

$$x_s \in \mathbb{Z}_+, \quad \forall s \in S.$$

结构识别

排班模型是集合覆盖的整数版本。时间段是元素，班次是候选集合，需求 d_t 可以大于 1。

排班例题

一天分为 4 个时间段，需求为 $d = (3, 5, 4, 2)$ 。有 3 种班次：

班次	时段 1	时段 2	时段 3	时段 4
A	1	1	0	0
B	0	1	1	0
C	0	0	1	1

设 x_A, x_B, x_C 为三个班次的人数。最小化总人数。

排班例题：模型答案

$$\min x_A + x_B + x_C$$

$$x_A \geq 3,$$

$$x_A + x_B \geq 5,$$

$$x_B + x_C \geq 4,$$

$$x_C \geq 2,$$

$$x_A, x_B, x_C \in \mathbb{Z}_+.$$

思考问题

若班次 A 成本更高，目标函数应如何修改？若每个班次最多安排 4 人，又应增加什么约束？

时间索引变量

当需要决定任务开始时间时，常用时间索引变量：

$$x_{jt} = \begin{cases} 1, & \text{任务} j \text{ 在时间} t \text{ 开始,} \\ 0, & \text{否则.} \end{cases}$$

每个任务恰好开始一次：

$$\sum_t x_{jt} = 1, \quad \forall j.$$

适用场景

机器调度、课程排课、直播活动排期、车辆装卸、会议室安排。

拓展：单资源时间冲突的直观含义

单资源指同一时刻只能处理一个任务的资源，例如一间会议室、一台机器、一个直播间或一辆装卸车。

例子

若任务 A 持续 2 个时间段，从 9 点开始，则它占用 9-10 和 10-11 两个时间段。此时另一个任务不能在 9 点或 10 点开始占用同一资源。

时间段	9-10	10-11	11-12
任务 A 从 9 点开始	占用	占用	
任务 B 从 10 点开始		占用	占用

冲突判断

如果两个被选择的任务在某个时间段同时占用同一资源，就发生时间冲突。

拓展：单资源时间冲突的约束

若任务 j 持续 p_j 个时间段，从 t 开始会占用

$$t, t+1, \dots, t+p_j-1.$$

因此，对每个被检查的时间段 τ ，需要统计所有“会占用 τ ”的任务开始选择：

$$\sum_j \sum_{t: t \leq \tau < t+p_j} x_{jt} \leq 1, \quad \forall \tau.$$

解释

$t \leq \tau < t+p_j$ 的意思是：任务 j 若从 t 开始，则时间段 τ 落在它的占用区间内。整条约束要求任意时间段内最多只有一个任务被安排在这项资源上。

练习 2：排班约束

某客服中心有早班、午班、晚班三类班次，分别覆盖如下时间段：

班次	9-12	12-15	15-18	18-21
早班	1	1	0	0
午班	0	1	1	0
晚班	0	0	1	1

若四个时间段需求为 4, 6, 5, 3，请写出需求覆盖约束。

时间维度建模要点

- ▶ 生产计划的核心是库存平衡约束；
- ▶ 固定开机成本需要 0-1 变量和启用约束；
- ▶ 排班模型本质上是带需求强度的覆盖模型；
- ▶ 时间索引变量可以表达任务开始时间；
- ▶ 时间冲突约束需要统计同一时间段被哪些任务占用。

目录

现代 **MIP** 求解器在做什么

结构化整数规划算法

本节主线：找解与证明最优

MIP 求解器同时做两件事：



找解

启发式、舍入、局部搜索和分支搜索用于尽快找到高质量整数可行解。

证明

LP 松弛、割平面和分支定界用于证明未搜索区域不可能产生更优解。

算法动机：枚举为什么不可行

整数规划最直接的想法是枚举所有整数解，再从中选择最优解。但可行组合数量通常随变量个数指数增长。

0-1 变量个数 n	20	40	60
可能组合数 2^n	1.05×10^6	1.10×10^{12}	1.15×10^{18}

直观计算

即使每秒检查 10^8 个方案，枚举 2^{60} 个方案也需要约 365 年。实际模型还要检查约束、计算目标值，通常更慢。

MIP 求解器模块：是什么、为什么、怎么做

模块	是什么	为什么要做	怎么做
Presolve	搜索前简化模型	减少变量、约束和无效分支	固定变量、收紧上下界、删除冗余约束
LP 松弛	暂时去掉整数性	给出界，判断子问题是否值得继续	解连续 LP ，得到松弛解和目标值界
割平面	加入有效不等式	让 LP 松弛 更接近整数可行域	识别被分数解违反的有效约束
分支定界	构造并搜索子问题树	处理分数变量并证明最优性	分支、求界、剪枝、更新 incumbent
启发式	快速找整数可行解	尽早得到好 incumbent ，帮助剪枝	舍入、修复、局部搜索、可行性泵

整体理解：这些模块不是顺序执行一次就结束，而是在搜索树的不同节点上反复交互。

手算例子：一个两变量整数规划

考虑最大化问题：

$$\max 5x_1 + 4x_2$$

$$6x_1 + 4x_2 \leq 24, \quad x_1 + 2x_2 \leq 6, \quad x_1, x_2 \in \mathbb{Z}_+.$$

先去掉整数性，求 LP 松弛。根节点 LP 松弛的一个最优解为

$$x_1 = 3, \quad x_2 = 1.5, \quad z_{LP} = 21.$$

课堂观察

这是最大化问题，因此根节点 LP 值 21 是整数最优值的上界。但 $x_2 = 1.5$ 不满足整数性，不能直接作为方案。

手算分支定界：根节点分支

根节点 LP 解中 $x_2 = 1.5$ ，因此可按 x_2 分支：

$$x_2 \leq 1 \quad \text{或} \quad x_2 \geq 2.$$

根节点： $x_1 = 3, x_2 = 1.5, z = 21$

左： $x_2 \leq 1$

右： $x_2 \geq 2$

分支目的

分支不是随意拆分，而是把分数解排除，同时保证所有整数可行解仍然被两个子问题覆盖。

手算分支定界：子节点求界

分别求两个子节点的 LP 松弛。

节点	LP 松弛最优解	结论
$x_2 \leq 1$	$x_1 = \frac{10}{3}, x_2 = 1, z = \frac{62}{3} \approx 20.67$	仍有分数变量，需要继续分支
$x_2 \geq 2$	$x_1 = 2, x_2 = 2, z = 18$	已是整数解，更新 incumbent 为 18

当前状态

最大化问题中，incumbent = 18 是下界；左节点 LP 上界约为 20.67，仍可能超过 18，因此不能剪枝。

手算分支定界：继续分支与证明

左节点中 $x_1 = \frac{10}{3}$ ，继续按 x_1 分支。

子节点	LP 松弛结果	处理
$x_2 \leq 1, x_1 \leq 3$	$x_1 = 3, x_2 = 1, z = 19$	整数解，更新 incumbent 为 19
$x_2 \leq 1, x_1 \geq 4$	$x_1 = 4, x_2 = 0, z = 20$	整数解，更新 incumbent 为 20

最终结论

当所有未处理节点的上界都不超过当前 incumbent 20 时，求解器证明 $x_1 = 4, x_2 = 0$ 是最优整数解。

案例：前置仓选址与客户分配

平台需要在候选前置仓 J 中选择若干个仓库，为社区 I 提供当日达服务。

- ▶ 开仓变量： $y_j \in \{0, 1\}$ ，表示是否启用仓库 j ；
- ▶ 分配变量： $x_{ij} \in \{0, 1\}$ ，表示社区 i 是否由仓库 j 服务；
- ▶ 开仓有固定成本 F_j ，分配有配送成本 c_{ij} ；
- ▶ 仓库容量为 Q_j ，社区需求为 d_i ；
- ▶ 若 $a_{ij} = 0$ ，表示仓库 j 不能服务社区 i 。

前置仓选址与客户分配: MILP 模型

$$\min \sum_{j \in J} F_j y_j + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

$$\sum_{j \in J} x_{ij} = 1, \quad \forall i \in I$$

$$\sum_{i \in I} d_i x_{ij} \leq Q_j y_j, \quad \forall j \in J$$

$$\sum_{j \in J} y_j \leq K, \quad x_{ij} \leq a_{ij}, \quad \forall i \in I, j \in J$$

$$x_{ij} \in \{0, 1\}, \quad y_j \in \{0, 1\}.$$

模型连接: 容量约束是典型启用约束。若 $y_j = 0$, 仓库 j 不能接收需求; 若 $y_j = 1$, 其服务总需求不能超过容量。

Gurobi 建模代码：变量与目标

```
import gurobipy as gp
from gurobipy import GRB

m = gp.Model("facility_location")

y = m.addVars(J, vtype=GRB.BINARY, name="open")
x = m.addVars(I, J, vtype=GRB.BINARY, name="assign")

m.setObjective(
    gp.quicksum(F[j] * y[j] for j in J)
    + gp.quicksum(c[i][j] * x[i, j] for i in I for j in J),
    GRB.MINIMIZE
)
```

对应模型

y_j 是开仓变量， x_{ij} 是分配变量；目标函数同时包含固定开仓成本和配送成本。

Gurobi 建模代码：约束与求解

```
for i in I:
    m.addConstr(gp.quicksum(x[i, j] for j in J) == 1)

for j in J:
    m.addConstr(
        gp.quicksum(d[i] * x[i, j] for i in I) <= Q[j] * y[j]
    )

m.addConstr(gp.quicksum(y[j] for j in J) <= K)

for i in I:
    for j in J:
        m.addConstr(x[i, j] <= a[i][j])

m.optimize()
```

Gurobi 日志：模型规模与根节点

以下为课堂示例日志片段，用于说明字段含义；具体数值会随数据、Gurobi 版本和参数设置变化。

```
Optimize a model with 54 rows, 45 columns and 130 nonzeros
```

```
Variable types: 0 continuous, 45 integer (45 binary)
```

```
Presolve removed 20 rows and 14 columns
```

```
Root relaxation: objective 3.286667e+02, 18 iterations
```

Nodes		Current Node		Objective Bounds				
Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	
0	0	328.66667	0	6	-	328.66667	-	

Gurobi 日志：搜索、启发式与割平面

H	0	0				372.00000	328.66667	11.65%
H	0	0				356.00000	328.66667	7.68%
	0	0	340.50000	0	4	356.00000	340.50000	4.35%
	3	1	348.00000	2	2	356.00000	348.00000	2.25%
*	5	0				352.00000	352.00000	0.00%

Cutting planes:

Gomory: 2

Cover: 3

MIR: 4

从日志回看 MIP 求解模块

日志信息	对应的算法含义
<code>Presolve removed ...</code>	预处理删除固定变量、冗余约束或不可服务组合
<code>Root relaxation</code>	根节点 LP 松弛，给出初始理论界
<code>IntInf</code>	当前 LP 解中仍违反整数性的变量数量
<code>H 或 *</code>	启发式或搜索过程找到新的整数可行解，更新 incumbent
<code>BestBd 与 Gap</code>	当前最优性证明的界和缺口
<code>Gomory, Cover, MIR</code>	求解器加入割平面，强化 LP 松弛

Gurobi 日志中的界：最小化问题

前置仓案例是最小化问题，因此日志中的界要按最小化方向理解。

日志字段	在最小化问题中的含义
Incumbent	当前找到的最好整数可行解，是目标值的上界
BestBd	当前所有未排除区域能达到的最好理论值，是目标值的下界
Gap	上界与下界之间的相对距离，衡量最优性尚未证明的空间
0.00%	上界与下界相等，当前整数解已被证明最优

课堂提醒

最大化问题中 LP 松弛通常给上界；最小化问题中 LP 松弛通常给下界。读日志时必须先看目标方向。

从模型到算法模块：前置仓案例

同一个 MILP 模型中的不同部分，会在求解器中触发不同的算法机制。

模型元素	求解器看到的困难	相关算法模块
$y_j \in \{0, 1\}$	开仓变量决定固定成本和可服务范围	分支、启发式、Presolve
$x_{ij} \in \{0, 1\}$	社区分配存在大量组合	LP 松弛、分支、可行解修复
$\sum_i d_i x_{ij} \leq Q_j y_j$	启用关系与容量共同作用	Presolve、割平面、界收紧
$x_{ij} \leq a_{ij}$	禁配关系可直接固定变量	Presolve 删除变量或收紧模型
$\sum_j y_j \leq K$	多个开仓决策被预算耦合	分支策略、Cover 类割平面

教学提示

求解器不是只“读公式”，而是在公式中寻找可推断、可放松、可分支、可加割的结构。

为什么 **gap** 可能下降很慢

求解日志中 **gap** 长时间不下降，通常不是单一原因造成的。

现象	可能原因
Incumbent 很久不变	启发式难以找到更好的整数可行解，或当前可行解附近改进空间小
BestBd 改善很慢	LP 松弛较弱，割平面和界收紧不足以快速缩小理论空间
节点数量增长快	分支后子问题仍然相似，剪枝能力不足
根节点 gap 很大	Big-M 过大、变量定义较弱，或缺少有效业务约束
日志出现数值提示	成本、容量、 Big-M 的量级差异过大，影响 LP 求解和割生成

通用 MIP 求解器的能力边界

Gurobi、CPLEX、SCIP 等求解器非常强大，但并不意味着建模者可以忽略问题结构。

求解器擅长

自动进行预处理、LP 松弛、割平面、分支搜索、启发式和并行搜索。

建模者负责

给出清晰变量、紧的上下界、合理 Big-M、避免无意义对称，并识别可利用的结构。

过渡

当变量或约束数量本身呈指数增长时，仅把完整模型交给通用求解器往往不够。这时需要结构化算法：动态生成变量、动态加入约束，或把大问题分解成若干子问题。

LP 松弛与上下界

以最大化问题为例：

$$z_{\text{IP}}^* = \max\{c^T x : Ax \leq b, x \in \mathbb{Z}^n\}.$$

去掉整数约束得到 LP 松弛：

$$z_{\text{LP}}^* = \max\{c^T x : Ax \leq b, x \in \mathbb{R}^n\}.$$

由于 LP 松弛可行域更大，

$$z_{\text{LP}}^* \geq z_{\text{IP}}^*.$$

对最大化问题

LP 松弛给出上界；任何整数可行解给出下界。

最大化与最小化中的界

LP 松弛的界方向取决于目标方向。

问题类型	LP 松弛	整数可行解
最大化	上界	下界
最小化	下界	上界

解释

LP 松弛扩大了可行域。最大化时目标值可能更高，因此给上界；最小化时目标值可能更低，因此给下界。

MIP Gap: 最优性缺口

求解器常用 **gap** 衡量当前解与最优性证明之间的距离。

以最小化问题为例，设当前最好整数可行解为 z_{inc} ，当前最好下界为 z_{bd} ，常见相对 **gap** 为

$$\text{gap} = \frac{|z_{inc} - z_{bd}|}{|z_{inc}| + \epsilon}.$$

gap 大

说明当前解和理论界之间距离较大，最优性尚未充分证明。

gap 小

说明当前整数解已经接近可证明最优。**gap** 为 0 时证明最优。

MIP Gap: 一个数值例子

考虑最小化问题。若当前最好整数可行解为

$$z_{\text{inc}} = 105,$$

当前最好下界为

$$z_{\text{bd}} = 100,$$

则相对 gap 约为

$$\frac{105 - 100}{105} \approx 4.76\%.$$

解释

这表示当前解至多比可证明的最优值差约 4.76%。若设置求解容差为 5%，求解器可以接受当前解；若要求严格最优，则还需继续证明。

Integrality Gap: 整数间隙

LP 松弛与整数最优值之间的差距称为整数间隙。以最大化问题为例：

$$z_{LP}^* \geq z_{IP}^*.$$

两者相差越明显，LP 松弛提供的界通常越弱。

建模含义

弱松弛会导致分支定界需要处理更多节点。紧的 Big-M、有效不等式和更合理的变量定义，都有助于缩小整数间隙。

松弛强弱	对求解的影响
LP 值接近整数最优	界更有效，搜索更快
LP 值远离整数最优	剪枝困难，搜索树变大

LP 松弛紧不紧：一个建模例子

考虑启用约束

$$0 \leq x \leq My, \quad y \in \{0, 1\}.$$

若实际业务上 $x \leq 120$ ，则 $M = 120$ 比 $M = 10^6$ 更紧。

写法	LP 松弛中的影响
$x \leq 10^6 y$	很小的分数 y 就能支持较大的 x ，松弛很弱
$x \leq 120 y$	分数 y 对 x 的限制更接近真实启用关系

结论

模型强弱不是抽象概念。紧的上界会直接改善 LP 松弛，从而帮助求解器剪枝。

三种剪枝情形

剪枝原因	含义
不可行剪枝	当前节点 LP 松弛不可行，因此没有整数可行解
界限剪枝	当前节点上界不可能超过已有最好整数解
整数剪枝	当前节点 LP 解已经是整数解，可更新最好解并停止该分支

理解

剪枝越有效，搜索树越小，求解速度通常越快。

割平面：把分数解切掉

割平面是在模型中加入新的线性约束，使得：

- ▶ 不排除任何整数可行解；
- ▶ 排除当前 LP 松弛中的某些分数解；
- ▶ 使 LP 松弛更接近整数可行域。

直觉

割平面不是随意增加限制，而是增加“所有整数解都满足”的有效约束。

例子：背包问题中的 **Cover Cut**

考虑背包约束

$$\sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

若某个物品集合 C 的总重量超过容量：

$$\sum_{i \in C} w_i > W,$$

则集合 C 中的物品不可能全部被选中，因此有有效不等式

$$\sum_{i \in C} x_i \leq |C| - 1.$$

直觉

Cover cut 不改变任何整数可行解，但可以切掉一些 LP 松弛中的分数解，是割平面思想在背包模型中的直观例子。

Presolve: 预处理

预处理会在正式搜索前简化模型，常见操作包括：

- ▶ 删除冗余约束；
- ▶ 固定必然取某个值的变量；
- ▶ 收紧变量上下界；
- ▶ 检测明显不可行；
- ▶ 合并重复或等价约束。

例子

若 $0 \leq x \leq 10y$ 、 $x \geq 5$ 、 $y \in \{0, 1\}$ ，则可推断 $y = 1$ 。预处理可以提前固定该变量，减少搜索规模。

Presolve: 一个推断例子

设

$$x_1 + x_2 + x_3 \leq 1, \quad x_i \in \{0, 1\}.$$

若模型中又有

$$x_1 + x_2 \geq 1,$$

则预处理可以推出：

$$x_3 = 0, \quad x_1 + x_2 = 1.$$

解释

前两条约束共同说明 x_1, x_2 中至少选一个，同时三个变量总共最多选一个，因此 x_3 不可能取 1。

启发式：先找到一个好整数解

整数规划求解器不仅证明最优性，也会主动寻找整数可行解。

启发式思想	基本含义
-------	------

舍入启发式	将 LP 分数解调整为整数解，再修复约束违反
-------	------------------------

Diving	沿着某些分数变量逐步固定，快速深入搜索树
--------	----------------------

局部搜索	在已有整数解附近交换、增删或调整变量
------	--------------------

作用

好的整数可行解可以提供更强的 **incumbent**，从而帮助分支定界更早剪枝。

时间限制下如何理解求解结果

实际问题常设置 **time limit**。求解器停止时，可能并未证明最优。

状态	含义
Optimal	已证明当前整数解最优
Feasible with gap	找到整数可行解，但尚有最优性缺口
Infeasible	已证明模型无可行解
Time limit	到达时间上限，需结合 incumbent 、 bound 和 gap 判断质量

求解慢：不一定是求解器差

同一个业务问题，不同建模方式会显著影响求解器表现。

常见原因	对求解的影响
Big-M 取值过大	LP 松弛很弱，分支树可能急剧增大
缺少合理上下界	Presolve 难以收紧模型，节点界质量较差
变量定义不合适	产生大量对称或等价方案，求解器重复搜索
数据尺度差异过大	数值稳定性变差，日志中可能出现较多数值警告
缺少有效业务逻辑	求解器需要自己发现本可由建模者直接写出的限制

本节小结：学生需要掌握什么

- ▶ 会区分“找到一个好整数解”和“证明它最优”；
- ▶ 知道 LP 松弛为什么提供界，并能判断最大化/最小化中的界方向；
- ▶ 能解释分支定界如何通过分支、求界和剪枝证明最优；
- ▶ 知道割平面切掉的是分数解，不应排除任何整数可行解；
- ▶ 能从 Gurobi 日志中读出 incumbent、best bound、gap、nodes 和 cuts；
- ▶ 理解建模质量会影响求解速度，尤其是 Big-M、上下界和变量定义。

一句话

MIP 求解器的核心不是“枚举所有方案”，而是在搜索过程中不断找好解、收紧界，并最终证明没有更好的整数解。

目录

现代 **MIP** 求解器在做什么

结构化整数规划算法

本节目标：先看例子，再记名字

有些整数规划模型直接交给求解器也可以求，但规模一大就会很慢。本节不要求大家掌握复杂算法细节，只需要理解三种基本想法：

遇到的困难	朴素想法
可选方案太多	先列出少量方案，不够再补
约束太多	先不写所有约束，发现违反再加
问题可以分层	先做关键决策，再检查后续安排

课堂要求

听懂“为什么这样拆”和“每一步大概做什么”即可；算法名称放在最后统一对应。

从一个直观问题开始

假设我们要做配送路线选择。客户很多，车辆有容量限制，每条路线必须从仓库出发再回到仓库。

直接建模

决定车辆走哪条边、先送谁、后送谁，再加容量、时间和子回路约束。

换个角度

先把一条完整可行路线看作一个方案，然后只决定选择哪些路线。

结构化思想

复杂性没有消失，而是被拆成两步：先生成合法方案，再从合法方案中选择组合。

例子一：路线选择的主模型

设 R 是已经列出的候选路线集合， $x_r = 1$ 表示选择路线 r 。

$$\begin{aligned} \min \quad & \sum_{r \in R} c_r x_r \\ \sum_{r \in R} a_{ir} x_r &= 1, \quad \forall i \in I \\ x_r &\in \{0, 1\}, \quad \forall r \in R. \end{aligned}$$

解释

$a_{ir} = 1$ 表示路线 r 服务客户 i 。约束要求每个客户恰好被一条选中路线服务。

例子一：路线从哪里来

如果所有路线都提前列出来，数量可能非常大。课堂上可以先只列出一些看起来合理的路线。

路线	服务客户	成本
r_1	1, 2	18
r_2	2, 3	20
r_3	3, 4	19
r_4	1, 4	22

怎么做

先用已有路线求一个模型；如果发现还需要更好的路线，就再生成一条新路线放进来。

例子一：补路线的思路

补路线时不需要一次生成所有路线，而是问一个更小的问题：

问题

能不能找到一条新的合法路线，使总成本有可能下降？

一条新路线必须同时满足：

- ▶ 从仓库出发并回到仓库；
- ▶ 车辆容量不超过上限；
- ▶ 若有时间窗，服务时间要合法；
- ▶ 成本相对当前方案有改进价值。

算法名称：这种“先少放一些变量，再逐步补变量”的思想叫做列生成。

例子二：下料问题也可以这样做

长度为 10 的原材料要切成若干短件，需求如下：

产品	长度	需求
A	6	8
B	4	10
C	3	12

关键观察

一根原材料有很多种切法。每一种切法都可以看作一个“方案”。

例子二：先列出几种切割模式

模式	A 件数	B 件数	C 件数	使用长度
1	1	1	0	10
2	1	0	1	9
3	0	2	0	8
4	0	1	2	10
5	0	0	3	9

令 x_r 表示使用模式 r 的次数。目标是少用原材料：

$$\min \sum_r x_r.$$

思考

如果这些模式不够好，就继续寻找新的切割模式。

例子二：下料问题的模型

使用已有切割模式时，需求约束可以写成：

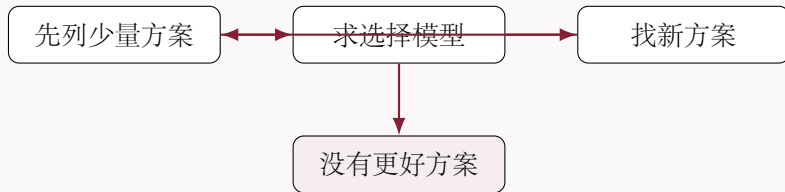
$$\sum_r a_{Ar}x_r \geq 8, \quad \sum_r a_{Br}x_r \geq 10, \quad \sum_r a_{Cr}x_r \geq 12.$$

$$x_r \in \mathbb{Z}_+.$$

怎么找新模式

找新切割模式，本质上是在一根长度为 10 的原材料里，选择若干产品，使这根材料“尽量有价值”。这和背包问题很像。

第一类方法小结：方案太多怎么办



记忆方式

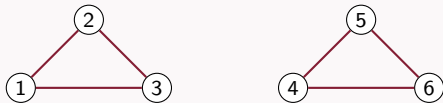
路线选择、下料、排班都可以理解为：变量不是一个小决策，而是一个完整方案。

例子三：约束太多怎么办

旅行商问题要求从一个城市出发，访问每个城市一次，并回到起点。

只写度约束的问题

如果只要求每个城市进一次、出一次，可能得到两个或多个小圈，而不是一条完整大回路。



两个小圈不是一条完整路线

例子三：发现小圈后再加约束

如果当前解中城市集合 S 自己形成一个小圈，就加入约束阻止它：

$$\sum_{i \in S} \sum_{j \in S, j \neq i} x_{ij} \leq |S| - 1.$$

解释

集合 S 内部最多只能选 $|S| - 1$ 条边，因此它不能自己闭合成一个小圈。

步骤	操作
----	----

- | | |
|---|------------------|
| 1 | 先求一个较小模型 |
| 2 | 检查是否出现小圈 |
| 3 | 若出现小圈，就加入排除小圈的约束 |
| 4 | 继续求解 |
-

第二类方法小结：约束太多怎么办

有些约束理论上很多，但实际求解过程中未必都用得上。

不推荐

一开始把所有可能约束都写出来，模型巨大，求解困难。

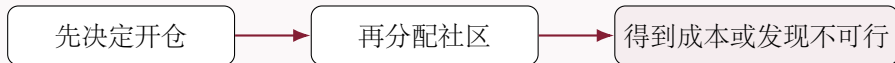
更自然

先求一个较小模型，看到违反哪条规则，再加入对应约束。

算法名称：这种“按需加入约束”的思想是割平面和 Branch-and-Cut 的核心。

例子四：先选址，再分配

考虑前置仓选址问题。平台先决定开哪些仓，再把社区分配给已开的仓。



关键观察

开仓变量 y_j 是战略决策；给定 y_j 后，社区分配问题会简单很多。

例子四：给定开仓方案后检查

假设有三个候选仓库 A、B、C。某一次主问题给出：

$$y_A = 1, \quad y_B = 0, \quad y_C = 1.$$

接下来只需要回答：

- ▶ A 和 C 的容量是否足够服务所有社区？
- ▶ 每个社区是否都有可达的已开仓库？
- ▶ 在这个开仓方案下，最低配送成本是多少？

反馈

如果不可行，就告诉主问题“这类开仓方案不行”；如果可行，就把真实配送成本反馈给主问题。

第三类方法小结：分层决策

层次	例子
第一层	是否开仓、是否建设、是否启用设备
第二层	客户分配、运输流量、生产数量、服务安排

算法名称

这种“主问题先做关键决策，子问题检查后续安排”的思想叫 **Benders 分解**。大二阶段只需要理解它是一种分层求解思想。

补充例子：背包问题可以逐步做

背包问题也可以看成“逐个物品做决定”。

$$\max \sum_i p_i x_i, \quad \sum_i w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

元素	在背包问题中的含义
----	-----------

阶段	已经考虑到第几个物品
----	------------

状态	当前还剩多少容量
----	----------

决策	第 i 个物品选或不选
----	---------------

算法名称：这种按阶段和状态递推的方法叫动态规划。

课堂练习：先判断困难在哪里

请先判断每个问题的主要困难。

编号	场景
1	下料问题中，可能的切割模式太多，无法全部列出来。
2	TSP 中，为了排除所有小圈，需要考虑大量城市子集。
3	选址问题中，先决定开哪些仓库，再安排社区分配。
4	背包问题中，可以按物品顺序和剩余容量逐步决策。

课堂练习：先判断困难在哪里

请先判断每个问题的主要困难。

编号	场景
1	下料问题中，可能的切割模式太多，无法全部列出来。
2	TSP 中，为了排除所有小圈，需要考虑大量城市子集。
3	选址问题中，先决定开哪些仓库，再安排社区分配。
4	背包问题中，可以按物品顺序和剩余容量逐步决策。

参考答案

1：方案太多；2：约束太多；3：分层决策；4：阶段与状态递推。

算法名称对照表

课堂直觉	常见算法名称
先列少量完整方案，不够再补方案	列生成；整数版本常叫 Branch-and-Price
先不写所有约束，发现违反再加约束	割平面；结合分支定界就是 Branch-and-Cut
先做关键 0-1 决策，再检查后续连续安排	Benders 分解
按物品、时间或容量状态逐步递推	动态规划

本节小结

- ▶ 结构化算法的核心不是背复杂术语，而是识别问题的困难来源；
- ▶ 如果变量代表完整方案，常用“先少列方案，再逐步补方案”的思想；
- ▶ 如果约束数量太多，常用“发现违反，再加入约束”的思想；
- ▶ 如果问题有先后层次，可以先做关键决策，再检查后续安排；
- ▶ 如果问题有自然阶段和状态，可以用递推方法逐步求解。

综合建模：从模块到完整模型

综合模型往往不是单一结构，而是多个基本结构的组合。



建模原则

先确定关键决策，再逐步加入业务规则。不要试图一次写完整模型。

综合案例：前置仓选址与服务分配

某零售平台计划从候选前置仓中选择若干个仓库，为多个社区提供当日达服务。

- ▶ 候选仓库集合为 J ，社区集合为 I ；
- ▶ 建设仓库 j 的固定成本为 F_j ，容量为 Q_j ；
- ▶ 社区 i 的日需求为 d_i ；
- ▶ 社区 i 分配给仓库 j 的配送成本为 c_{ij} ；
- ▶ 距离过远的仓库不能服务该社区；
- ▶ 最多建设 K 个仓库。

综合案例：变量定义

建设变量：

$$y_j = \begin{cases} 1, & \text{建设前置仓}j, \\ 0, & \text{不建设前置仓}j, \end{cases} \quad j \in J.$$

分配变量：

$$x_{ij} = \begin{cases} 1, & \text{社区}i \text{ 分配给前置仓}j, \\ 0, & \text{否则,} \end{cases} \quad i \in I, j \in A_i.$$

其中 $A_i \subseteq J$ 表示社区 i 可选择的仓库集合。

综合案例：目标函数

目标是最小化固定建设成本和配送服务成本。

$$\min \sum_{j \in J} F_j y_j + \sum_{i \in I} \sum_{j \in A_i} c_{ij} x_{ij}.$$

固定建设成本

$$\sum_{j \in J} F_j y_j$$

只有建设仓库时发生。

配送服务成本

$$\sum_i \sum_j c_{ij} x_{ij}$$

由社区分配方案决定。

综合案例：分配与启用约束

每个社区恰好分配给一个可服务仓库：

$$\sum_{j \in A_i} x_{ij} = 1, \quad \forall i \in I.$$

只有建设仓库，社区才能分配给该仓库：

$$x_{ij} \leq y_j, \quad \forall i \in I, j \in A_i.$$

最多建设 K 个仓库：

$$\sum_{j \in J} y_j \leq K.$$

综合案例：容量约束

前置仓 j 的服务需求总量不能超过容量：

$$\sum_{i \in I: j \in A_i} d_i x_{ij} \leq Q_j y_j, \quad \forall j \in J.$$

未建设

$$y_j = 0 \Rightarrow \sum_i d_i x_{ij} \leq 0.$$

不能服务任何社区。

已建设

$$y_j = 1 \Rightarrow \sum_i d_i x_{ij} \leq Q_j.$$

服务需求不超过容量。

综合案例：完整模型

$$\min \sum_{j \in J} F_j y_j + \sum_{i \in I} \sum_{j \in A_i} c_{ij} x_{ij}$$

$$\sum_{j \in A_i} x_{ij} = 1, \quad \forall i \in I,$$

$$x_{ij} \leq y_j, \quad \forall i \in I, j \in A_i,$$

$$\sum_{i \in I: j \in A_i} d_i x_{ij} \leq Q_j y_j, \quad \forall j \in J,$$

$$\sum_{j \in J} y_j \leq K,$$

$$x_{ij}, y_j \in \{0, 1\}.$$

综合案例：逐条解释

约束	现实含义
$\sum_{j \in A_i} x_{ij} = 1$	每个社区恰好由一个可服务仓库负责
$x_{ij} \leq y_j$	未建设仓库不能接收社区分配
$\sum_i d_i x_{ij} \leq Q_j y_j$	仓库服务需求不得超过容量
$\sum_j y_j \leq K$	建设仓库数量不超过预算或管理上限

扩展一：加入服务覆盖率

若要求至少 90% 的需求由 3 公里内仓库服务，设 $N_i \subseteq A_i$ 为社区 i 的近距离仓库集合。

$$\sum_{i \in I} d_i \sum_{j \in N_i} x_{ij} \geq 0.9 \sum_{i \in I} d_i.$$

解释

左侧是由近距离仓库服务的需求量；右侧是总需求的 90%。这是服务水平约束。

扩展二：加入配送班次

若仓库还需要安排早班、午班和晚班，设

$$z_{js} = \begin{cases} 1, & \text{仓库}j \text{ 安排班次}s, \\ 0, & \text{否则.} \end{cases}$$

未建设仓库不能安排班次：

$$z_{js} \leq y_j, \quad \forall j, s.$$

若仓库建设，则至少安排一个班次：

$$\sum_s z_{js} \geq y_j, \quad \forall j.$$

扩展三：加入路线选择

若每个仓库可选择若干配送路线，设 $r \in R_j$ 表示仓库 j 的候选路线。

$$u_{jr} = \begin{cases} 1, & \text{选择仓库 } j \text{ 的路线 } r, \\ 0, & \text{否则.} \end{cases}$$

每个社区至少被一条选中路线覆盖：

$$\sum_j \sum_{r \in R_j} a_{ijr} u_{jr} \geq 1, \quad \forall i.$$

只有建设仓库，才能选择该仓库路线：

$$u_{jr} \leq y_j, \quad \forall j, r.$$

综合案例结构总结

模型模块	对应约束
选址	$y_j \in \{0, 1\}$
分配	$\sum_j x_{ij} = 1$
启用关系	$x_{ij} \leq y_j$
容量	$\sum_i d_i x_{ij} \leq Q_j y_j$
服务覆盖率	$\sum_i d_i \sum_{j \in N_i} x_{ij} \geq \alpha \sum_i d_i$

关键能力

看到复杂问题时，把它拆成若干熟悉模块，再逐步组合。

练习 4：综合建模

某连锁咖啡品牌计划选择若干门店作为即时配送门店，并决定每个订单区域由哪家门店服务。

- ▶ 每家门店启用有固定成本和最大订单处理能力；
- ▶ 每个订单区域必须分配给一家可达门店；
- ▶ 最多启用 5 家门店；
- ▶ 至少 80% 的订单量要由 2 公里内门店服务。

任务

写出变量定义，并列出具目标函数、分配约束、启用约束、容量约束和服务水平约束。

模型检查清单

检查项	问题
变量定义	下标、取值范围、现实含义是否清楚?
目标函数	每一项单位是否一致? 方向是否正确?
覆盖或分配	是至少一次、恰好一次还是至多一次?
启用约束	未启用时相关变量是否被迫为 0?
容量上界	Big-M 是否来自实际上界?

知识点回顾

- ▶ 集合覆盖、划分、打包是许多 0-1 模型的基础结构；
- ▶ 生产计划依赖库存平衡，固定开机成本引入混合整数变量；
- ▶ 排班问题可看作带需求强度的覆盖模型；
- ▶ 分支定界通过 LP 松弛提供界，通过分支和剪枝搜索整数解；
- ▶ 割平面可以加强模型，切掉分数解但保留整数解；
- ▶ 综合建模应把复杂问题拆成选址、分配、容量、覆盖和逻辑模块。

综合建模作业

请选择一个运营问题，建立整数规划模型。可选主题包括：

- ▶ 前置仓选址与社区分配；
- ▶ 客服班次排班；
- ▶ 广告档期选择；
- ▶ 配送路线选择；
- ▶ 生产计划与开机安排。

作业要求：

1. 明确下标集合和变量定义；
2. 写出目标函数；
3. 至少包含 4 类约束；
4. 逐条解释约束的现实含义；
5. 指出模型中最可能影响求解速度的地方。